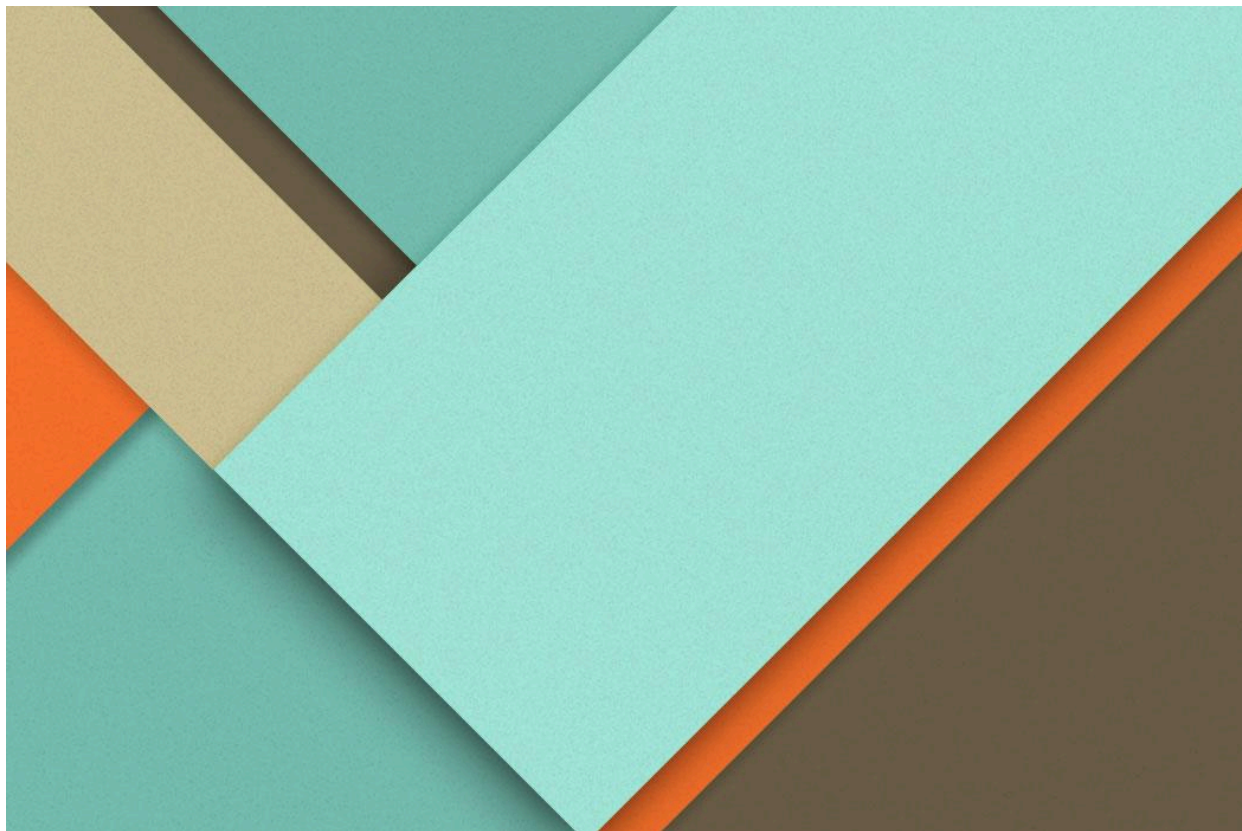




UD5: Manual Técnico Hospital

04/05/2026

—

José Manuel Sánchez Rosal

2º DAM

IES Antonio Gala

1400 Palma del Río (Córdoba)

Índice

Índice.....	1
1. Arquitectura de Modelos y Base de Datos (ORM).....	1
1.1. Modelo: HospitalEspecialidad.....	2
1.2. Modelo: HospitalMedico.....	3
1.3. Modelo: HospitalCita.....	4
2. Diseño de Vistas e Interfaz (XML).....	5
2.1. Especialidad.....	6
2.1.1. Especialidad-Search (mejora).....	6
2.1.2. Especialidad-Lista (V2).....	6
2.1.3. Especialidad-Kanban (V2).....	7
2.1.4. Especialidad-Formulario (V2).....	9
2.2. Médico.....	10
2.2.1. Médico-Search (nueva).....	10
2.2.2. Médico-Lista (V2).....	11
2.2.3. Médico-Kanban (V2).....	12
2.2.4. Médico-Form (V2).....	14
2.3. Cita.....	15
2.3.1. Cita-Search (nueva).....	15
2.3.2. Cita-Lista (V2).....	16
2.3.3. Cita-Form (V2).....	16
2.4. Ventanas de Acción.....	17
2.5. Menús.....	18
3. Gestión de Permisos y Accesos.....	19
4. Documentación y Estándares.....	19

1. Arquitectura de Modelos y Base de Datos (ORM)

El núcleo del módulo hospitalario se ha desarrollado utilizando el ORM (Object-Relational Mapping) de Odoo, mapeando las clases de Python directamente a tablas de PostgreSQL. Se han aplicado estándares de documentación Google Docstrings para facilitar el mantenimiento y la lectura del código fuente por parte de futuros desarrolladores.

1.1. Modelo: HospitalEspecialidad

Este modelo actúa como tabla maestra para categorizar a los facultativos y gestionar la logística física (consultas) del hospital:

```
# -*- coding: utf-8 -*-

from odoo import models, fields

class HospitalEspecialidad(models.Model):
    """
    Modelo que representa las especialidades médicas del hospital.
    Permite categorizar a los médicos y asignarles un número de
    consulta
    físico dentro del centro.

    Atributes:

        name (fields.Char): Nombre de la especialidad (Max. 40
        caracteres).

        consulta (fields.Selection): Menú desplegable para asignar la
        sala de consulta (1 a 5).

        medico_ids (fields.One2many): Relación inversa con los
        médicos asignados a esta especialidad.
    """

    _name = 'hospital.especialidad'
    _description = 'Especialidades del Hospital'
    name = fields.Char(string='Nombre', required=True, size=40)
    consulta = fields.Selection([
        ('consulta1', 'Consulta 1'),
        ('consulta2', 'Consulta 2'),
        ('consulta3', 'Consulta 3'),
        ('consulta4', 'Consulta 4'),
```

```

        ('consulta5', 'Consulta 5'),
    ], string='Consultas', default='consulta1')
    medico_ids = fields.One2many('hospital.medico',
    'especialidad_id', string='Médicos en esta especialidad')

```

1.2. Modelo: HospitalMedico

Representa la entidad central del módulo. Implementa restricciones de tamaño en los campos de texto, validaciones de tipo fecha y campos binarios para el tratamiento de imágenes ([fields.Image](#)). Incorpora relaciones [Many2one](#) para heredar la especialidad y [One2many](#) para visualizar de forma inversa las citas programadas:

```
class HospitalMedico(models.Model):
```

```
    """
```

```
    Modelo que gestiona el personal facultativo del hospital.
```

```
    Almacena los datos personales, de contacto y perfil profesional
    de cada médico.
```

```
    Se relaciona bidireccionalmente con las especialidades y las
    citas de los pacientes.
```

Attributes:

```
        name (fields.Char): Nombre y apellidos completos del médico
        (Max. 40 caracteres).
```

```
        domicilio (fields.Char): Dirección postal de residencia.
```

```
        telefono (fields.Char): Número de teléfono de contacto (9
        dígitos).
```

```
        fecha_ingreso (fields.Date): Fecha de contratación o alta en
        el sistema.
```

```
        image (fields.Image): Fotografía de perfil, utilizada para
        renderizar la vista Kanban.
```

```
        especialidad_id (fields.Many2one): Relación con el modelo
        hospital.especialidad.
```

```
        cita_ids (fields.One2many): Relación inversa con el modelo
        hospital.cita.
```

```
    """
```

```
_name = 'hospital.medico'
_description = 'Médicos del Hospital'
name = fields.Char(string='Nombre y Apellidos', required=True,
size=40)
domicilio = fields.Char(string='Domicilio', required=True,
size=40)
telefono = fields.Char(string='Teléfono', required=True, size=9)
fecha_ingreso = fields.Date(string='Fecha de Ingreso',
required=True)
image = fields.Image(string="Foto de Perfil")
especialidad_id = fields.Many2one('hospital.especialidad',
string='Especialidad', required=True)
cita_ids = fields.One2many('hospital.cita', 'medico_id',
string='Citas Asignadas')
```

1.3. Modelo: HospitalCita

Modelo transaccional orientado al registro de eventos temporales. Utiliza campos de tipo [Datetime](#) para asegurar la precisión horaria y cruza la información entre las entidades independientes de pacientes y el personal médico:

```
class HospitalCita(models.Model):
```

```
    """
```

```
    Modelo transaccional para la programación de citas médicas.
```

```
    Vincula a un paciente (entidad de texto) con un facultativo
    específico
```

```
    en una fecha y hora determinadas.
```

Attributes:

`paciente (fields.Char):` Nombre y apellidos del paciente que solicita la cita.

`medico_id (fields.Many2one):` Médico asignado para pasar la consulta.

`fecha_cita (fields.Datetime):` Marca de tiempo (fecha y hora) programada para la visita.

"""

`_name = 'hospital.cita'`

`_description = 'Citas de Pacientes'`

`paciente = fields.Char(string='Paciente (Nombre y Apellido)', required=True, size=40)`

`medico_id = fields.Many2one('hospital.medico', string='Médico', required=True)`

`fecha_cita = fields.Datetime(string='Fecha y hora de la cita', required=True)`

2. Diseño de Vistas e Interfaz (XML)

Las interfaces de usuario se han definido de manera declarativa mediante XML. Se ha adaptado el código a la sintaxis estricta de Odoo 18, sustituyendo las etiquetas obsoletas `<tree>` por `<list>`. Se han implementado a posteriori, mejoras en todas las vistas usando **Bootstrap**. El desarrollo incluye la implementación de bloques `<kanban>` avanzados que hacen uso de directivas QWeb (`t-att-src`) para renderizar dinámicamente datos binarios almacenados en base de datos:

2.1. Especialidad

2.1.1. Especialidad-Search (mejora)

- Se incorpora una vista de búsqueda personalizada. Además de permitir la búsqueda por nombre y número de consulta, se añade un filtro de agrupación (`group_by`) que permite categorizar la vista lista basándose en la consulta asignada:

```

<odoo>
    <record id="view_hospital_especialidad_search"
model="ir.ui.view">
        <field name="name">hospital.especialidad.search</field>
        <field name="model">hospital.especialidad</field>
        <field name="arch" type="xml">
            <search>
                <field name="name"/>
                <field name="consulta"/>
                <group expand="0" string="Agrupar por">
                    <filter string="Consulta"
name="group_by_consulta" context="{ 'group_by': 'consulta' }"/>
                </group>
            </search>
        </field>
    </record>

```

2.1.2 Especialidad-Lista (V2)

- Se aplican decoradores visuales a la etiqueta `<list>`. El nombre de la especialidad se renderiza en negrita (`decoration-bf`) y el campo consulta se estiliza mediante un componente visual tipo etiqueta (`widget="badge"`):

```

<record id="view_hospital_especialidad_list" model="ir.ui.view">
    <field name="name">hospital.especialidad.list</field>
    <field name="model">hospital.especialidad</field>
    <field name="arch" type="xml">
        <list decoration-info="consulta">
            <field name="name" string="Especialidad"
decoration-bf="1"/>
            <field name="consulta" optional="show" widget="badge"
decoration-info="1"/>
        </list>
    </field>
</record>

```

```

    </field>
</record>

```

2.1.3. Especialidad-Kanban (V2)

- Se ha rediseñado completamente la vista mediante el uso de clases de Bootstrap (**d-flex, text-center, p-4**) para lograr un diseño responsivo. Se implementa lógica condicional QWeb (**<t t-if>**) para renderizar iconos de FontAwesome de diferentes colores según la especialidad médica leída de la base de datos:

```

<record id="view_hospital_especialidad_kanban"
model="ir.ui.view">
    <field name="name">hospital.especialidad.kanban</field>
    <field name="model" >hospital.especialidad</field>
    <field name="arch" type="xml">
        <kanban class="o_kanban_mobile" sample="1">
            <field name="name"/>
            <templates>
                <t t-name="kanban-box">
                    <div t-attf-class="oe_kanban_global_click
oe_kanban_card p-4 d-flex flex-column justify-content-center
text-center" style="min-height: 200px;">
                        <div class="o_kanban_record_top mb-3">
                            <div
class="o_kanban_record_headings">
                                <div class="mb-3">
                                    <t
t-if="record.name.raw_value == 'Cardiologia' or record.name.raw_value
== 'Cardiología'">
                                        <i class="fa fa-heartbeat
fa-4x text-danger"/>
                                    </t>
                                </div>
                            </div>
                        </div>
                    </div>
                </t>
            </templates>
        </kanban>
    </field>
</record>

```



```

                                <t
t-elif="record.name.raw_value == 'Nutricion' or record.name.raw_value
== 'Nutrición'">

                                <i class="fa fa-leaf
fa-4x text-success"/>

                                </t>

                                <t
t-elif="record.name.raw_value == 'Aparato Locomotor' or
record.name.raw_value == 'Aparato locomotor'">

                                <i class="fa
fa-wheelchair fa-4x text-warning"/>

                                </t>

                                <t
t-elif="record.name.raw_value == 'Psicologia' or
record.name.raw_value == 'Psicología'">

                                <i class="fa
fa-puzzle-piece fa-4x text-info"/>

                                </t>

                                <t t-else="">

                                <i class="fa fa-user-md
fa-4x text-primary"/>

                                </t>
                                </div>
                                <strong
class="o_kanban_record_title fs-3 text-dark">
                                <field name="name"/>
                                </strong>
                                </div>
                                </div>
                                <div class="o_kanban_record_bottom
mt-auto">

                                <div class="oe_kanban_bottom_center">

```

```

                                <span class="badge rounded-pill
text-bg-info fs-5 px-3 py-2">
                                <i class="fa fa-door-open
me-2"/>
                                <field name="consulta"/>
                                </span>
                            </div>
                        </div>
                    </div>
                </t>
            </templates>
        </kanban>
    </field>
</record>

```

2.1.4. Especialidad-Formulario (V2)

- Se estructura la cabecera mediante la clase `oe_title`. El selector de consultas se mejora a nivel de usabilidad transformándolo en un conjunto de botones de radio horizontales (`widget="radio"` `options="{ 'horizontal': true }"`):

```

<record id="view_hospital_especialidad_form" model="ir.ui.view">
    <field name="name">hospital.especialidad.form</field>
    <field name="model">hospital.especialidad</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <div class="oe_title">
                    <label for="name" class="oe_edit_only"
string="Nombre de Especialidad"/>
                    <h1>
                        <field name="name" placeholder="Ej.
Cardiología"/>

```

```

        </h1>
    </div>
    <group>
        <group>
            <field name="consulta" widget="radio"
options="{ 'horizontal': true }"/>
        </group>
    </group>
</sheet>
</form>
</field>
</record>

```

2.2. Médico

2.2.1. Médico-Search (nueva)

- Se amplía el dominio de búsqueda nativo. El atributo `filter_domain` permite que al escribir en la barra superior se busque simultáneamente tanto en el nombre del facultativo como en su número de teléfono. Se añade agrupación por especialidad:

```

<record id="view_hospital_medico_search" model="ir.ui.view">
    <field name="name">hospital.medico.search</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
        <search>
            <field name="name" filter_domain="['|', ('name',
'ilike', self), ('telefono', 'ilike', self)]"/>
            <field name="especialidad_id"/>
            <group expand="0" string="Agrupar por">

```

```

        <filter string="Especialidad"
name="group_by_especialidad"
context="{ 'group_by': 'especialidad_id' }"/>
    </group>
</search>
</field>
</record>

```

2.2.2. Médico-Lista (V2)

- La tabla se enriquece incrustando la fotografía del médico mediante el widget `image` con dimensiones fijadas como avatar (`oe_avatar`). Se incluyen widgets específicos para formatear el teléfono (`phone`) y etiquetas de color (`decoration-primary`) para la especialidad:

```

<record id="view_hospital_medico_list" model="ir.ui.view">
    <field name="name">hospital.medico.list</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
        <list>
            <field name="image" widget="image" class="oe_avatar"
options="{ 'size': [35, 35] }"/>
            <field name="name" decoration-bf="1"/>
            <field name="especialidad_id" widget="badge"
decoration-primary="1"/>
            <field name="telefono" widget="phone"/>
            <field name="fecha_ingreso" optional="hide"/>
        </list>
    </field>
</record>

```

2.2.3. Médico-Kanban (V2)

- Se utiliza la clase estructural avanzada `o_kanban_record_has_image_fill` combinada con `o_kanban_image_fill_left`. Esto permite que la imagen binaria del

médico ocupe proporcionalmente todo el margen izquierdo de la tarjeta Kanban. Se añaden iconos informativos para los datos de contacto:

```
<record id="view_hospital_medico_kanban" model="ir.ui.view">
  <field name="name">hospital.medico.kanban</field>
  <field name="model">hospital.medico</field>
  <field name="arch" type="xml">
    <kanban sample="1">
      <field name="id"/>
      <field name="image"/>
      <field name="name"/>
      <field name="especialidad_id"/>
      <field name="telefono"/>
      <templates>
        <t t-name="kanban-box">
          <div class="oe_kanban_global_click
o_kanban_record_has_image_fill p-2" style="min-height: 180px;">
            <t t-if="record.image.raw_value">
              <div class="o_kanban_image_fill_left
d-none d-md-block"
t-attf-style="background-image:url('#{kanban_image('hospital.medico',
'image', record.id.raw_value)}'); width: 140px;"/>
            </t>
            <t t-else="">
              <div class="o_kanban_image"
style="width: 140px;"/>
            </t>
            <div class="oe_kanban_details d-flex
flex-column justify-content-between p-3">
              <div>
```

```

                                <strong
class="o_kanban_record_title fs-3 text-dark"><field
name="name"/></strong>

                                <div class="mt-2">

                                    <span class="badge
text-bg-primary fs-6 px-3 py-2"><field
name="especialidad_id"/></span>

                                </div>
                            </div>

                            <div class="o_kanban_record_bottom
mt-3">

                                <div class="oe_kanban_bottom_left
text-muted fs-5">

                                    <i class="fa fa-phone me-2
text-info"/> <field name="telefono" widget="phone"/>

                                </div>

                            </div>

                        </div>

                    </div>

                </t>

            </templates>

        </kanban>

    </field>

</record>

```

2.2.4. Médico-Form (V2)

- La vista de edición integra un previsualizador de imagen ([preview_image](#)). Los campos de datos se distribuyen ordenadamente utilizando etiquetas [<group>](#) anidadas, e implementan los widgets [selection](#), [phone](#) y [date](#) para asegurar la correcta entrada de datos:

```
<record id="view_hospital_medico_form" model="ir.ui.view">
```

```

    <field name="name">hospital.medico.form</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <field name="image" widget="image"
class="oe_avatar" options="{ 'preview_image': 'image' }"/>
                <div class="oe_title">
                    <label for="name" class="oe_edit_only"
string="Nombre y Apellidos"/>
                    <h1>
                        <field name="name" placeholder="Dr. /
Dra."/>
                    </h1>
                </div>
                <group>
                    <group>
                        <field name="especialidad_id"
widget="selection"/>
                        <field name="telefono" widget="phone"/>
                    </group>
                    <group>
                        <field name="domicilio"/>
                        <field name="fecha_ingreso"
widget="date"/>
                    </group>
                </group>
            </sheet>
        </form>
    </field>

```

```
</record>
```

2.3. Cita

2.3.1. Cita-Search (nueva)

- Se introduce un dominio dinámico (`context_today().strftime('%Y-%m-%d')`) en los filtros para aislar con un solo clic las 'Citas Futuras'. Además, permite agrupar los registros por Médico o por el día exacto de la cita

```
<record id="view_hospital_cita_search" model="ir.ui.view">
  <field name="name">hospital.cita.search</field>
  <field name="model">hospital.cita</field>
  <field name="arch" type="xml">
    <search>
      <field name="paciente"/>
      <field name="medico_id"/>
      <filter string="Citas Futuras" name="citas_futuras"
domain="[( 'fecha_cita', '>=',
context_today().strftime('%Y-%m-%d'))]" />
      <group expand="0" string="Agrupar por">
        <filter string="Médico" name="group_by_medico"
context="{ 'group_by': 'medico_id' }" />
        <filter string="Fecha" name="group_by_fecha"
context="{ 'group_by': 'fecha_cita:day' }" />
      </group>
    </search>
  </field>
</record>
```


2.3.2. Cita-Lista (V2)

- Se fuerza la ordenación descendente por defecto (`default_order="fecha_cita desc"`). Se implementa un sistema de semaforización mediante decoradores condicionales a nivel de lista: las citas programadas en fecha actual o futura se resaltan en verde (`decoration-success`), mientras que las vencidas se marcan en rojo (`decoration-danger`):

```
<record id="view_hospital_cita_list" model="ir.ui.view">
  <field name="name">hospital.cita.list</field>
  <field name="model">hospital.cita</field>
  <field name="arch" type="xml">
    <list default_order="fecha_cita desc"
decoration-success="fecha_cita >= current_date"
decoration-danger="fecha_cita <= current_date">
      <field name="fecha_cita" widget="datetime"/>
      <field name="paciente" decoration-bf="1"/>
      <field name="medico_id" widget="many2one_avatar"/>
    </list>
  </field>
</record>
```

2.3.3. Cita-Form (V2)

- Destaca la inclusión del componente visual `<widget name="web_ribbon">` que simula una cinta de estado en la esquina del formulario. Para la selección del médico, se utiliza `many2one_avatar`, mostrando la foto del facultativo directamente en el desplegable de asignación:

```
<record id="view_hospital_cita_form" model="ir.ui.view">
  <field name="name">hospital.cita.form</field>
  <field name="model">hospital.cita</field>
  <field name="arch" type="xml">
    <form>
```

```
<sheet>

    <widget name="web_ribbon" title="Programada"
bg_color="text-bg-success"/>

    <div class="oe_title">

        <label for="paciente" class="oe_edit_only"
string="Paciente (Nombre y Apellido)"/>

        <h1>

            <field name="paciente"
placeholder="Nombre completo"/>

        </h1>

    </div>

    <group>

        <group>

            <field name="medico_id"
widget="many2one_avatar"/>

        </group>

        <group>

            <field name="fecha_cita"
widget="datetime"/>

        </group>

    </group>

</sheet>

</form>

</field>

</record>
```

2.4. Ventanas de Acción

- Las ventanas de acción (`ir.actions.act_window`) actúan como el motor que vincula los modelos de datos con la interfaz de usuario. En este módulo, se han configurado cuidadosamente para definir el comportamiento de apertura de cada entidad. Se utiliza el atributo `view_mode` para establecer la jerarquía de visualización, priorizando la vista **Kanban** en Médicos y Especialidades por su alto valor gráfico, mientras que en Citas se prioriza la vista **Lista** para facilitar la gestión de la agenda. Además, se definen mensajes de ayuda (`help`) para guiar al usuario en caso de que no existan registros creados:

```
<record id="action_hospital_especialidad"
model="ir.actions.act_window">
    <field name="name">Especialidades</field>
    <field name="res_model">hospital.especialidad</field>
    <field name="view_mode">kanban,list,form</field>
</record>

<record id="action_hospital_medico"
model="ir.actions.act_window">
    <field name="name">Médicos</field>
    <field name="res_model">hospital.medico</field>
    <field name="view_mode">kanban,list,form</field>
</record>

<record id="action_hospital_cita" model="ir.actions.act_window">
    <field name="name">Citas</field>
    <field name="res_model">hospital.cita</field>
    <field name="view_mode">list,form</field>
</record>
```

2.5. Menús

- La arquitectura de menús define la navegación del sistema de forma jerárquica y organizada. Se ha implementado un menú raíz principal que actúa como contenedor en la barra superior de Odoo. A partir de este, se despliegan tres submenús vinculados a sus respectivas ventanas de acción. Se hace uso del atributo **sequence** para garantizar un orden lógico de trabajo: primero la configuración de la estructura (**Especialidades**), segundo el alta del personal (**Médicos**) y, finalmente, la operativa diaria (**Citas**). Esta disposición asegura que el flujo de trabajo sea intuitivo y profesional para el personal del centro:

```
<menuitem id="menu_hospital_root" name="Hospital"/>

<menuitem id="menu_hospital_especialidades" name="Especialidades"
parent="menu_hospital_root" action="action_hospital_especialidad"
sequence="10"/>

<menuitem id="menu_hospital_medicos" name="Médicos"
parent="menu_hospital_root" action="action_hospital_medico"
sequence="20"/>

<menuitem id="menu_hospital_citas" name="Citas"
parent="menu_hospital_root" action="action_hospital_cita"
sequence="30"/>

</odoo>
```

3. Gestión de Permisos y Accesos

El acceso a las estructuras de datos está regido por las directivas del archivo `ir.model.access.csv`. Se ha configurado un perfil base que otorga permisos CRUD completos (Create = 1, Read = 1, Update = 1, Delete = 1) vinculados al grupo `base.group_user`, asegurando que el personal administrativo pueda operar el módulo sin restricciones técnicas:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
```



```
access_hospital_especialidad,hospital.especialidad,model_hospital_especialidad,base.group_user,1,1,1,1
```

```
access_hospital_medico,hospital.medico,model_hospital_medico,base.group_user,1,1,1,1
```

```
access_hospital_cita,hospital.cita,model_hospital_cita,base.group_user,1,1,1,1
```

4. Documentación y Estándares

Todo el código Python ha sido documentado mediante la extensión [autoDocstring](#), garantizando que cada atributo y clase contenga su descripción técnica integrada. Los nombres de variables, las relaciones y los decoradores cumplen con las directrices PEP8 y los estándares de desarrollo oficiales de Odoo.